



Informe del Proyecto: Servicio Técnico

Arquitectura de microservicios con Java, Spring Boot y PostgreSQL

JULIO DE 2026

ACTUALIZADO HASTA EL PUNTO 5

Resumen Ejecutivo

El sistema

Backend construido con **Java 25** y **Spring Boot 4.1.0** para gestionar el ciclo completo de una atención técnica: clientes, equipos, tickets, diagnósticos, repuestos, cotizaciones y notificaciones.

Arquitectura actual

9 microservicios + 1 BFF. Ocho servicios utilizan su propia base PostgreSQL. Notificación no persiste datos: simula el envío y deja evidencia en logs.

Novedad principal

El microservicio de **cotización** consulta ticket y repuesto, calcula mano de obra, descuento e IVA, y descuenta inventario solo al aprobar.

- ❏ Implementado hasta el punto 5. Gateway, BFF ampliado, seguridad transversal, pruebas finales y Docker Compose completo están pendientes.

Introducción y Contexto

En un servicio técnico real, la información pasa por varias personas y etapas: el cliente entrega un equipo, se abre una solicitud, un técnico revisa la falla, se requieren repuestos, se calcula el costo y se informa al cliente. Cuando estos datos se manejan en planillas separadas o mensajes informales, es fácil perder el historial o duplicar información.

Este proyecto resuelve ese problema mediante una plataforma modular que separa autenticación, clientes, empleados, tickets, equipos, repuestos, diagnósticos, notificaciones y cotizaciones. Esta separación facilita entender el código, probar cada parte y realizar cambios sin afectar innecesariamente al resto del sistema.



Objetivos y Alcance

1

Objetivo general

Desarrollar un backend de microservicios con APIs REST, autenticación JWT, persistencia independiente y comunicación entre servicios.

2

Objetivos específicos

- Registrar clientes, empleados y técnicos.
- Crear, actualizar y filtrar tickets.
- Controlar inventario sin stock negativo.
- Generar cotizaciones con precios reales y descontar stock al aprobar.
- Simular notificaciones sin base de datos.

3

Alcance actual

Llega hasta el **punto 5**: los cinco microservicios nuevos tienen su lógica principal. La integración completa del BFF, Gateway, seguridad transversal y cobertura de pruebas pertenecen a etapas posteriores.

Arquitectura del Sistema

La arquitectura sigue el enfoque de microservicios y el patrón **Database per Service**. El BFF actúa como capa de entrada para consultas que combinan información de varios servicios. La autenticación se basa en **JWT**: el token se reenvía en el encabezado Authorization durante las llamadas internas entre cotización, ticket y repuesto.

Responsabilidad única

Cada servicio atiende un dominio concreto.

DB por servicio

Sin tablas compartidas; comunicación solo por APIs.

REST síncrono

Integraciones actuales mediante HTTP.

Stateless

JWT viaja en cada solicitud, sin sesión en memoria.

Tecnologías Utilizadas

Tecnología	Uso en el proyecto
Java 25	Lenguaje principal
Spring Boot 4.1.0	Creación de microservicios
Spring Security + JWT	Autenticación stateless
PostgreSQL	Persistencia por servicio
Flyway	Migraciones de esquemas
Gradle	Compilación y ejecución
Docker / Docker Compose	Contenedores de bases y servicios
JUnit, Mockito, JaCoCo	Pruebas y cobertura
Actuator / Swagger	Salud y documentación de APIs



Microservicios y Responsabilidades

Servicio	Puerto	Responsabilidad	Persistencia
BFF	8080	Centraliza y combina información	Sin DB
Auth	7778	Registro, login y JWT	auth_db
Empleado	8081	Gestiona empleados y técnicos	employee_db
Ticket	8082	Estados, prioridades y asignaciones	ticket
Cliente	8083	Datos de clientes	cliente_db
Equipo	8084	Equipos asociados a clientes	equipo_db
Repuesto	8085	Inventario y stock no negativo	repuesto_db
Diagnóstico	8086	Diagnósticos por ticket y empleado	diagnostico_db
Notificación	8087	Simula envíos, registra en logs	Sin DB
Cotización	8088	Costos, stock y aprobación	cotizacion_db

Integración: Cotización, Ticket y Repuesto

Cotización es el primer flujo de negocio que requiere comunicación directa con otros servicios, evitando duplicar precio o stock.



Fórmula de cálculo

subtotal = manoObra + costoRepuestos

neto = subtotal - descuento

IVA = neto × 0,19

total = neto + IVA

Decisión de diseño

El precio del repuesto no lo envía el cliente: cotización lo obtiene desde el microservicio repuesto, manteniendo una **única fuente de verdad** para el inventario.

Bases de Datos y Ejecución

Puertos por servicio

Servicio	Base	Local
Ticket	ticket	5433
Cliente	cliente_db	5434
Auth	auth_db	5435
Empleado	employee_db	5436
Cotización	cotizacion_db	5437
Equipo	equipo_db	5438
Repuesto	repuesto_db	5439
Diagnóstico	diagnostico_db	5440

Ejecución local (ejemplo: cotización)

```
docker compose up -d db-cotizacion
cd cotizacion
.\gradlew.bat bootRun
```

Comprobación de salud

```
Invoke-RestMethod http://localhost:8088/actuator/health
# Respuesta esperada: status UP
```

Compilación

```
.\gradlew.bat clean build
```

❏ Hibernate usa ddl-auto=validate: valida tablas contra entidades pero no reemplaza Flyway. En Docker Compose, usar nombre del servicio y puerto 5432.

Calidad, Pruebas y Seguridad

Herramientas de prueba

El proyecto utiliza **JUnit** y **Mockito** para pruebas unitarias e integración, y **JaCoCo** para medir cobertura.

```
.\gradlew.bat test  
.\gradlew.bat test jacocoTestReport
```

Meta: cobertura mínima del **40 %**. Las pruebas deben cubrir: stock insuficiente, validación de entradas, creación, aprobación y rechazo de cotizaciones, y manejo de respuestas erróneas.

Seguridad actual

La autenticación JWT ya forma parte del sistema. Cotización reenvía el encabezado **Authorization** a ticket y repuesto para conservar el contexto de seguridad.

Pendiente (punto 8)

Aplicación transversal de seguridad, manejo uniforme de errores y Javadoc en los cinco servicios nuevos.



El sistema tiene base funcional, pero no se considera terminado hasta validar seguridad, errores, documentación, pruebas, Docker Compose integral y logging centralizado.

Estado Actual y Trabajo Pendiente

✅ Implementado hasta el punto 5

- CRUD de equipos, repuestos y diagnósticos con bases propias.
- Notificaciones simuladas sin base de datos.
- CRUD de cotizaciones con cálculos automáticos.
- Integración cotización ↔ ticket ↔ repuesto.
- Validación de stock y reducción al aprobar.
- Migraciones Flyway y diagramas C2/C3 actualizados.

❑ Pendiente

- **Punto 6:** API Gateway como puerta de entrada única.
- **Punto 7:** Ampliar BFF con endpoints de orquestación.
- **Punto 8:** Seguridad, manejo de errores y Javadoc en los cinco servicios nuevos.
- **Punto 9:** Pruebas y cobertura JaCoCo ≥ 40 %.
- **Punto 10:** Docker Compose integral con todos los servicios.
- **Punto 11:** Logging con GlitchTip y endpoint /api/auth/me.

Conclusión

El proyecto Servicio Técnico ya supera un CRUD básico. La arquitectura representa un proceso real completo: registro de equipo, creación de ticket, diagnóstico técnico, consulta de repuestos, generación de cotización y ajuste de stock al aprobar.

Base sólida

PostgreSQL por servicio, Flyway para migraciones y JWT para autenticación aplican prácticas habituales en backends modernos.

Modularidad

La separación por microservicios permite que cada módulo evolucione de manera independiente sin afectar al resto.

Próximos pasos

Cerrar la arquitectura: Gateway, BFF, seguridad, pruebas, Docker Compose completo y logging centralizado.

